ASSIGNMENT 2 – CO2

TASK 04

Q4 - Comparing HashMap, TreeMap and LinkedHashMap for Inventory Management

Course: CSA0903 – Programming in Java

Problem Context: Railway e-Ticket Generation

Name: Sowmiya. D

Register Number: 192511063

Aim

To compare HashMap, TreeMap and LinkedHashMap based on insertion, searching, iteration, memory usage characteristics and report ordering in an inventory management system.

1.1 Working Principle

- **HashMap:** Stores key-value pairs using hashing. It provides average O(1) insertion, searching and deletion. It does not guarantee ordering.
- **TreeMap:** Stores keys in sorted order using a Red-Black Tree. Insertion, searching and deletion take O(log n).
- **LinkedHashMap:** Maintains insertion order using a linked list along with hashing. Its basic operations take average O(1).

1.2 Test Data

The experiment uses three inventory sizes:

Test	Number of items
Small	1,000
Medium	10,000
Large	50,000

Each inventory item contains an item code and item details. The program performs 100,000 lookups for each map and records execution time.

1.3 Performance Comparison

Operation	HashMap	TreeMap	LinkedHashMap
Insert	O(1) average	O(log n)	O(1) average

Search	O(1) average	O(log n)	O(1) average
Delete	O(1) average	O(log n)	O(1) average
Iteration	O(n)	O(n)	O(n)
Ordering	No guaranteed order	Sorted by key	Insertion order

1.4 Report Ordering

- HashMap does not guarantee a particular order. Its keys must be sorted separately when a sorted report is required.
- TreeMap automatically maintains items in ascending key order.
- LinkedHashMap displays items in the order in which they were inserted.

For an inventory report that must always display item codes alphabetically, TreeMap is a suitable choice.

1.5 Alternative – Hashtable

Hashtable is an older synchronized Map implementation. It does not allow null keys or null values. Although it provides thread-safe individual operations, modern applications generally use ConcurrentHashMap when concurrent access is required.

1.6 Alternative – Sorted Array

A sorted array can store item codes in ascending order and use binary search for lookup.

- Searching: O(log n)
- Insertion: O(n), because elements may need shifting
- Deletion: O(n), because elements may need shifting
- Ordered traversal: O(n)

A sorted array is useful when data changes infrequently, but a Map is more convenient for frequently updated inventory records.

1.7 Conclusion

HashMap is suitable for fast inventory lookups when ordering is not required. TreeMap is appropriate when inventory reports must remain sorted by item code. LinkedHashMap is useful when the original insertion order must be maintained. The selection depends on the application's ordering and performance requirements.

2. Java program – Main.java

Create this file inside Assignments 02\Task04.

The program:

- Creates HashMap, TreeMap and LinkedHashMap inventories.
- Tests 1,000, 10,000 and 50,000 items.
- Measures insertion time.
- Performs 100,000 lookups for each map.
- Measures iteration time.
- Displays a sample of each map's ordering.

Coding:

```
import java.util.*;

public class Main {

    static final int LOOKUPS = 100_000;
    static final int REPEATS = 5;

    static String itemCode(int number) {
        return String.format("ITEM%06d", number);
    }

    static String itemName(int number) {
        return "Product-" + number;
    }

    static long measureLookup(Map<String, String> map, int size) {
        long checksum = 0;
        long start = System.nanoTime();
```

```
    for (int i = 0; i < LOOKUPS; i++) {
        int number = (i % size) + 1;
        String value = map.get(itemCode(number));

        if (value != null) {
            checksum += value.length();
        }
    }

    long end = System.nanoTime();

    // Prevent the lookup result from being unused.
    if (checksum == -1) {
        System.out.println(checksum);
    }

    return end - start;
}

static long measureIteration(Map<String, String> map) {
    long checksum = 0;
    long start = System.nanoTime();

    for (Map.Entry<String, String> entry : map.entrySet()) {
        checksum += entry.getKey().length();
    }
```

```
    long end = System.nanoTime();

    if (checksum == -1) {
        System.out.println(checksum);
    }

    return end - start;
}

static void testInventory(int size) {

    Map<String, String> hashMap = new HashMap<>();
    Map<String, String> treeMap = new TreeMap<>();
    Map<String, String> linkedHashMap = new LinkedHashMap<>();

    long start;
    long hashInsert;
    long treeInsert;
    long linkedInsert;

    // Measure insertion time.
    start = System.nanoTime();
    for (int i = 1; i <= size; i++) {
        hashMap.put(itemCode(i), itemName(i));
    }
    hashInsert = System.nanoTime() - start;
```

```
start = System.nanoTime();
for (int i = 1; i <= size; i++) {
    treeMap.put(itemCode(i), itemName(i));
}
treeInsert = System.nanoTime() - start;

start = System.nanoTime();
for (int i = 1; i <= size; i++) {
    linkedHashMap.put(itemCode(i), itemName(i));
}
linkedInsert = System.nanoTime() - start;

// Warm-up lookup runs.
measureLookup(hashMap, size);
measureLookup(treeMap, size);
measureLookup(linkedHashMap, size);

long hashLookup = 0;
long treeLookup = 0;
long linkedLookup = 0;

for (int i = 0; i < REPEATS; i++) {
    hashLookup += measureLookup(hashMap, size);
    treeLookup += measureLookup(treeMap, size);
    linkedLookup += measureLookup(linkedHashMap, size);
}
```

```
// Measure iteration time.
long hashIteration = measureIteration(hashMap);
long treeIteration = measureIteration(treeMap);
long linkedIteration = measureIteration(linkedHashMap);

System.out.println("\nInventory size: " + size);
System.out.println("Times are in milliseconds.");

System.out.printf(
    "HashMap      - Insert: %.3f, Average lookup: %.3f, Iteration: %.3f%n",
    hashInsert / 1_000_000.0,
    hashLookup / REPEATS / 1_000_000.0,
    hashIteration / 1_000_000.0
);

System.out.printf(
    "TreeMap      - Insert: %.3f, Average lookup: %.3f, Iteration: %.3f%n",
    treeInsert / 1_000_000.0,
    treeLookup / REPEATS / 1_000_000.0,
    treeIteration / 1_000_000.0
);

System.out.printf(
    "LinkedHashMap- Insert: %.3f, Average lookup: %.3f, Iteration: %.3f%n",
    linkedInsert / 1_000_000.0,
    linkedLookup / REPEATS / 1_000_000.0,
```

```
        linkedIteration / 1_000_000.0
    );

    // Display first five item codes to compare ordering.
    System.out.println("\nFirst five keys in each map:");

    System.out.println("HashMap: " + firstFive(hashMap));
    System.out.println("TreeMap: " + firstFive(treeMap));
    System.out.println("LinkedHashMap: " + firstFive(linkedHashMap));
}

static List<String> firstFive(Map<String, String> map) {
    List<String> result = new ArrayList<>();
    int count = 0;

    for (String key : map.keySet()) {
        result.add(key);
        count++;

        if (count == 5) {
            break;
        }
    }

    return result;
}
```

```
    public static void main(String[] args) {

        int[] sizes = {1000, 10000, 50000};

        System.out.println("INVENTORY MAP PERFORMANCE TEST");
        System.out.println("Lookups per map: " + LOOKUPS);
        System.out.println("Lookup repetitions: " + REPEATS);

        for (int size : sizes) {
            testInventory(size);
        }

        System.out.println("\nAll inventory tests completed.");
    }
}
```

Output:

```
INVENTORY MAP PERFORMANCE TEST
Lookups per map: 100000
Lookup repetitions: 5

Inventory size: 1000
Times are in milliseconds.
HashMap      - Insert: ..., Average lookup: ..., Iteration: ...
TreeMap      - Insert: ..., Average lookup: ..., Iteration: ...
LinkedHashMap- Insert: ..., Average lookup: ..., Iteration: ...
```

First five keys in each map:

HashMap: [...]

TreeMap: [ITEM000001, ITEM000002, ITEM000003, ITEM000004, ITEM000005]

LinkedHashMap: [ITEM000001, ITEM000002, ITEM000003, ITEM000004, ITEM000005]

... (10,000 and 50,000 item tests)

All inventory tests completed.

Implementation:

